

Lab 16:

OK, 現在你應該知道繼承不能被濫用。你也知道以下的程式碼該怎麼改。

不過其實一直還有一個更複雜的問題，那就是以下濫用繼承的程式碼為什麼不好。它會有什麼問題？也就是說，你可能以這個繼承來造條條，但是你好像也沒有發生過什麼問題不是嗎？

(此題會深刻思考題，比重等同其他題目的 3 倍) 請仔細思考。

```
class EmployeeComus: public ListContainer {
public:
    ...
    // public routines
    void addNewEmployee( Employee employee );
    void RemoveEmployee( Employee employee );

    Employee newInitInList();
    Employee firstItem();
    Employee lastItem();
    ...
private:
    ...
};
```

Answer of class 2019

- 繼承是物件導向中最強烈的耦合，容易牽一髮而動全身，當更變父類別時將同步影響到所有子類別，而且亂用繼承的話，很可能導致a繼承b，b繼承c，.....最後變得很很難維護，並有很多凌在的問題。此種寫法若ListContainer被更改了，那麼EmployeeCensus將受到嚴重的影響，所以在合此使用合成的has-a關係會較恰當。像是我們先寫了一個冷氣類別，接下來要做一個汽車的類別，我們不會因為汽車有冷氣，所以就繼承冷氣，而是使用合成較正確。
- EmployeeCensus不應繼承ListContainer，因為EmployeeCensus不屬於ListContainer，不符合繼承的基本原則；這種寫法在程式執行上不會有問題，但是在程式碼的邏輯上則有很大的缺陷，會使其他人混淆
- Employee並不是ListContainer所延伸出來的東西，所以不應該這樣使用
- 因為我們之所以要用OO就是為了解決維護的問題，也就是日後會比較容易看得懂，把一個很複雜的問題，抽象化成好幾個class，有點類似系統化的感覺，這樣的套用卻會反而造成不易閱讀的混亂，這樣就違背了我們當初之所以要用OO的初衷，如果只是單單要使用它，那大可別的方式，method, global function之類的

Answer of class 2019

- 這樣繼承listcontainer
employee並沒有使用到listContainer原本的函數
應該宣告一個listContainer來存(employee)
- 例子中的Employee並不是listContainer的一種，所以想要使用listContainer中的method，必須自己create一個listContainer的object去使用那些method
- employee繼承 listcontainer 但是沒有用到listContainer的函數，卻還需要自己定義類似功能的function 容易造成混淆而且會產生很多沒有用到的程式碼，加上如果之後有其他類別繼承employee 會使代碼變得很多
- 直接將listContainer定義成EmployCensus的member variable。
private:
listContainer lc;
- 不需要繼承
只要在class內建立Employee的變數
太多的繼承層次會使程式執行效能變慢
- 會造成programmer的混亂，之後要擴充或維護也不容易
- 這種程式碼基本上只是做一個介面接上原本物件，本質上並沒有減少多少代碼，反而沒有減少代碼的可讀性
- 這樣會讓以後接手的人無法簡單明白這個class的作用 後續可能會有應用或誤用
-

The left is called inheritance, the right is called composition

```
class employeeCensus: public vector {  
    // public routine  
    void AddEmployee(Employee employee);  
    void RemoveEmployee(Employee employee);  
  
    Employee NextItemInList();  
    Employee FirstItem();  
    Employee LastItem();  
  
};  
  
class employeeCensus {  
    // employeeCensus member data  
    vector employees;  
  
    // public routine  
    void AddEmployee(Employee employee);  
    void RemoveEmployee(Employee employee);  
  
    Employee NextItemInList();  
    Employee FirstItem() { return employees.front(); }  
    Employee LastItem();  
  
};
```

Bad 1: you inherit all the method and interfaces of vector

- For example, vector has the following methods
- Can I override them?
- What if some stupid subclass overrides front carelessly?
- By the way, I can **ANY** of the methods inherited from vector, **such as clear()** to wipe out all the contents of the vector.
- A class only carry the methods needed

6

Bad 2, polymorphism freedom can kick your ass

```
Vector *v = new employeecensus();
```

This is allowed !! And why do you want to do that?



7

Remember: when you try to find answer for OOAD problem it is always about **FUTURE**

8

Bad 3:

9

Bad 4:

10